

Daily Macro Brief Agent - 1-Page Memo

Design Tradeoffs

This prototype is designed to run manually or through a scheduler on an always-on machine. I chose *Gmail SMTP* rather than *Telegram* because I feel email fits the intended daily workflow better. *Gemini 2.5 Flash-Lite* is the default LLM because the model's job is narrow at this stage: synthesize structured inputs into a concise macro brief. Code, rather than the LLM, owns market data, tables, charting, source links, validation, logging, and delivery.

The core design choice is to separate facts from writing. Market numbers come from public APIs or scraping: *Yahoo Finance chart data*, *Japan MOF JGB data*, *Frankfurter FX reference rates*, and *CoinGecko BTC data*. If live market data is unavailable and no cached data exists, the value cells are left blank intentionally. Calendar rows come from the *Fair Economy / Forex Factory weekly feed*; RSS sources feed *Theme Radar*. The LLM receives only structured facts and is validated for JSON shape, word limits, market-number consistency, source reuse, and portfolio logic before output is accepted.

The brief is tailored to a macro PM. **Dashboard** is a table with “Reading”. **The 3 Things That Matter Today** is exactly three items with portfolio implications. **One Chart Worth Seeing** uses *USD/JPY* because the assumed book is long *USD/JPY*; the current chart has about three months of history and highlights the latest five observations. **Theme Radar** uses curated RSS sources with source-depth labels, so readers can see when the agent is using feed-level text rather than full article text. Beyond that, I added **Feedback Questionnaire**, **Source Status**, and **Assumptions** sections for future extension and completeness.

Scheduling was tested: *GitHub manual send* worked, *GitHub scheduled triggers* were unreliable in short-window tests, and a *MacBook launchd* scheduled send succeeded.

Position and Theme Assumptions

Because the PDF does not reveal the real book, the prototype stores explicit assumptions in the portfolio input file, *positions.csv*, under `inputs/portfolio/`. The assumed book is long *USD/JPY*, overweight *gold*, and exposed to an *EM debt basket*. The assumed themes are *higher-for-longer rates*, *fiscal pressure and term premium*, *China demand weakness*, and *policy divergence* across major central banks. Position rows are effective-date updates, so the latest prior row carries forward until changed or closed.

Three V2 Features Not Completed

1. Add paid *market data providers* so the agent is less dependent on free public data feeds.
2. Expand *Theme Radar* from RSS-level text to full-text reading for research notes, speeches, and selected newsletters (additional subscription required). At that point, a more advanced LLM may be more useful.
3. Use feedback history and richer book information to improve the relevance of the content and personalization of the brief over time.

One-Month Full-Time Roadmap

With one month full-time, I would turn this into a production briefing system: add stronger robustness checks, improve data-source quality, set up schedule alerting, add richer article ingestion, build feedback-driven improvement, and choose a cleaner deployment target such as a VPS or cloud scheduler. I would also add a daily evaluation report showing source freshness, failed fetches, token use, and runtime. As the system runs, I would continuously monitor whether the delivered brief meets all formatting and factual guardrails.

Actual Hours Spent

Building the agent was relatively fast; I spent about 2-3 hours on the initial scaffold. I then spent another full day, more than 10 additional hours, checking that the content was real, updated, reader-friendly, well-structured, and easier to extend later.